

P_WebStore - Rapport

Auteur : Thomas Reynaud

Ecole : ETML

Année : 2026

Durée : 24 périodes

Table des matières

1. [Introduction](#)
 2. [Environnement et outils](#)
 3. [Analyse initiale](#)
 4. [Implémentations obligatoires](#)
 5. [Implémentations optionnelles](#)
 6. [Résultats et validation](#)
 7. [Conclusion](#)
 8. [Livrables](#)
-

Introduction

Ce rapport documente le travail réalisé dans le cadre du projet individuel « **P_WebStore** ». L'objectif principal est de sécuriser une application web existante en implémentant les bonnes pratiques issues du **Top 10 OWASP 2025**, afin de régler les failles de sécurité les plus courantes.

Le projet dure **24 périodes** et inclut des activités obligatoires ainsi que des activités à choix pour atteindre un total de **15 points**. Chaque activité a été validée par l'enseignant après implémentation.

Environnement et outils

2.1 Environnement de développement

Composant	Détail
Système d'exploitation	Windows 10 / Ubuntu (WSL2)
Runtime	Node.JS v20.x
Conteneurisation	Docker Desktop
Éditeur	Visual Studio Code
Gestionnaire de versions	Git / GitHub
Base de données	MySQL (via Docker)

2.2 Dépôt GitHub

Le code source est versionné sur un fork du dépôt original. Un `.gitignore` prévient le versioning des `node_modules` et des données sensibles (fichier `.env`). Un historique Git complet documente chaque implémentation.

Dépôt : https://github.com/ETML-INF/secured_webshop

Analyse initiale de l'application

3.1 Présentation de l'application

L'application fournie est un web shop basique permettant la gestion de produits et d'utilisateurs. Elle utilise **Express.js** comme framework web et une base de données **MySQL**

3.2 Failles de sécurité identifiées

- Mots de passe stockés en clair dans la base de données
 - Requêtes SQL vulnérables aux injections
 - Absence d'authentification et de gestion des sessions
 - Pas de séparation des rôles
 - Aucune validation des entrées utilisateur
 - Communication non chiffrée
 - Pas de protection contre les attaques par force brute
-

Implémentations obligatoires

4.1 Page de login (Frontend)

Description : Création d'une interface de connexion permettant à l'utilisateur de saisir ses identifiants.

Implémentation : Une page HTML responsive (`login.html`) avec formulaire et intégration JavaScript pour appel API POST afin de se connecter et par la suite obtenir un token d'authentification.

Statut : Complété

4.2 Page d'inscription (Frontend)

Description : Création d'une interface d'inscription permettant aux nouveaux utilisateurs de créer un compte.

Implémentation : Une page HTML (`register.html`) avec validation côté client et appel API POST qui permet de créer un compte dans la base de donnée et bien sur le mot de passe de notre compte et hasher grâce a Bcrypt même si maintenant Bcrypt est obsolète et qu'il faudrait utiliser Argon2 selon le Top Ten Owasp.

Statut : Complété

4.3 Hashage des mots de passe

Description : Les mots de passe ne doivent jamais être stockés en clair. Nous utilisons **bcrypt** pour le hashage sécurisé.

Implémentation :

```
const hashedPassword = await bcrypt.hash(passwordPepper, 10);
```

- Utilisation de bcrypt avec 10 rounds de hachage
- Chaque hash est unique (contient un sel aléatoire)
- Coût computationnel : ~100ms par hash (ralentit les attaques par brute force)
- passwordPepper est le poivre de notre mdp il est stocké dans notre .env(environnement)

Statut : Complété

4.4 Ajout d'un sel (Salt)

Description : Un sel est une valeur aléatoire ajoutée au mot de passe avant le hashage pour renforcer la sécurité. Cela prévient les attaques par rainbow tables. (Rainbow Tables sont des tables avec des hash de mots connus qui servent à retrouver plus simplement des mot de passes à partir de leurs hashes)

Implémentation :

- Bcrypt gère automatiquement la génération et le stockage du sel
- 10 rounds = 2^{10} itérations de hachage
- Chaque utilisateur a un sel unique

Statut : Complété

4.5 Ajout d'un poivre (Pepper)

Description : Un poivre est une constante secrète ajoutée au mot de passe (contrairement au sel qui est aléatoire). Il est stocké en variable d'environnement, pas en base de données.

Implémentation :

```
const passwordPepper = password + process.env.SECRETKEY;  
const hashedPassword = await bcrypt.hash(passwordPepper, 10);
```

- Stocké dans le fichier .env
- Appliqué avant le hashage bcrypt
- Augmente la sécurité même si la base de données est compromise

Statut : Complété

4.6 Prévention des injections SQL

Description : Les injections SQL permettent à un attaquant d'exécuter du code SQL malveillant notamment dans des formulaire (Exemple : Login ou Contact). Nous utilisons des **requêtes paramétrées** pour prévenir cette vulnérabilité.

Exemple de vulnérabilité:

```
// VULNÉRABLE
const query = `SELECT * FROM users WHERE username = '${username}'`;
db.query(query, (err, results) => {
  // Si username = ' OR '1'='1', le SQL devient : WHERE username = '' OR '1'='1'
});
```

Correction:

```
// SÉCURISÉ
const query = "SELECT * FROM users WHERE username = ?";
db.query(query, [username], (err, results) => {
  // Les ? sont remplacés de manière sûre
});
```

Statut : Complété

4.7 Authentification par token JWT

Description : JWT (JSON Web Token) permet une authentification **stateless** sécurisée sans sessions serveur.

Implémentation :

```
const token = jwt.sign(
  { id: user.id, username: user.username, role: user.role },
  process.env.SECRETKEY,
  { expiresIn: "24h" },
);
```

Fonctionnement :

1. L'utilisateur se connecte avec ses identifiants
2. Le serveur crée un JWT signé avec sa clé secrète
3. Le token est envoyé au client et stocké localement
4. À chaque requête, le token est inclus dans le header **Authorization: Bearer <token>**
5. Le serveur vérifie la signature et l'expiration du token

Statut : Complété

4.8 Gestion des rôles dans le JWT

Description : Les rôles (`admin`, `user`) sont inclus dans le JWT et utilisés pour contrôler l'accès aux routes.

Implémentation :

Middleware d'authentification :

```
jwt.verify(token, process.env.SECRETKEY, (err, user) => {  
  if (err) {  
    return res.status(403).json({ error: "Token invalide ou expiré." });  
  }  
  req.user = user;  
  next();  
});
```

Middleware de vérification de rôle :

```
module.exports = (req, res, next) => {  
  if (req.user.role !== "admin") {  
    return res.status(403).json({ error: "Accès refusé." });  
  }  
  next();  
};
```

Application sur les routes :

```
app.use("/api/admin", verifyToken, isAdmin, adminRoute);
```

Statut : Complété

Implémentations optionnelles

5.1 HTTPS (Tâche 9)

Description : Le protocole HTTPS chiffre la communication entre le client et le serveur, empêchant l'interception des données et les attaques man-in-the-middle (qui fonctionne avec un attaquant qui s'interpose secrètement entre deux parties qui croient communiquer directement l'une avec l'autre).

Implémentation :

```
const https = require("https");  
const fs = require("fs");
```

```
const httpsOptions = {
  key: fs.readFileSync(path.join(__dirname, "server.key")),
  cert: fs.readFileSync(path.join(__dirname, "server.cert")),
};

https.createServer(httpsOptions, app).listen(443, () => {
  console.log("Serveur sécurisé lancé sur https://localhost:443");
});
```

Certificats :

- Auto-signés générés pour le développement
- En production, utiliser Let's Encrypt ou un CA de confiance

Statut : Complété

5.2 Politique de mot de passe fort

Description : Les mots de passe faibles sont une source majeure de vulnérabilités. Nous enforceons une politique stricte avec les règles ci-dessous.

Règles implémentées :

- Minimum **8 caractères**
- Au moins **1 majuscule** (A-Z)
- Au moins **1 minuscule** (a-z)
- Au moins **1 chiffre** (0-9)
- Au moins **1 caractère spécial** (!@#\$%^&*)

Implémentation :

```
const strongPasswordRegex =
  /^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[A-Za-z0-9]).{8,}$/;

if (!strongPasswordRegex.test(password)) {
  return res.status(400).json({
    error:
      "Le mot de passe doit contenir au moins 8 caractères, une majuscule,
      une minuscule, un chiffre et un caractère spécial.",
  });
}
```

Interface utilisateur :

- Un indicateur visuel côté client montre la **force du mot de passe** en temps réel
- Feedback instantané : Rouge → Orange → Vert

Statut : Complété et validé

5.3 Limitation des tentatives de login (Tâche 15)

Description : Le **rate limiting** protège contre les attaques par force brute en limitant le nombre de tentatives de connexion.

Implémentation :

```
const loginLimiter = rateLimit({
  windowMs: 1 * 60 * 1000, // 1 minute
  max: 5, // 5 tentatives max
  message: {
    error:
      "Trop de tentatives de connexion. Veuillez réessayer dans une minute.",
  },
});

app.use("/api/auth/login", loginLimiter);
```

Fonctionnement :

- 5 tentatives par minute
- Après 5 tentatives, le serveur retourne une erreur **429 (Too Many Requests)**
- Le compteur se réinitialise après 1 minute

Statut : Complété et validé

5.4 Verrouillage de compte (Tâche 16)

Description : Après **N tentatives de connexion échouées**, le compte est verrouillé pour une période définie pour ralentir les attaques par brute force au niveau de l'application.

Configuration :

```
MAX_LOGIN_ATTEMPTS=3      # Nombre max de tentatives avant verrouillage
LOCKOUT_MINUTES=15        # Durée du verrouillage en minutes
```

Colonnes base de données :

- **failed_login_attempts** (INT) : Compteur de tentatives échouées
- **lockout_until** (DATETIME) : Date/heure jusqu'à laquelle le compte est verrouillé

Implémentation :

```
if (lockoutUntil && lockoutUntil > now) {
  return res.status(423).json({
```

```
        error: `Compte verrouillé. Réessayez après  
        ${lockoutUntil.toLocaleString()}`  
    });  
}  
  
if (!isPasswordValid) {  
    const updatedAttempts = failedAttempts + 1;  
  
    if (updatedAttempts >= MAX_LOGIN_ATTEMPTS) {  
        const unlockDate = new Date(Date.now() + LOCKOUT_MINUTES * 60 * 1000);  
        // Verrouiller le compte  
    }  
  
    db.query(updateQuery, [updatedAttempts, lockoutValue, user.id], ...);  
}
```

Flux :

1. Tentative échouée -> `failed_login_attempts++`
2. Après 3 tentatives échouées -> Compte verrouillé pendant 15 minutes
3. `lockout_until` est défini à `NOW() + 15 minutes`
4. À la tentative suivante, si `lockout_until > NOW()`, accès refusé
5. Après 15 minutes, le verrouillage expire et le compteur se réinitialise

Statut : Complété

5.5 Gestion des erreurs

Description : Les erreurs ne doivent jamais divulguer d'informations sensibles en production.

Implémentation :

```
// VULNÉRABLE  
if (err) {  
    return res.status(500).json({ error: err.message }); // Expose les  
    détails  
}  
  
// SÉCURISÉ  
if (err) {  
    console.error("Erreur DB:", err); // Log détaillé en serveur  
    return res.status(500).json({ error: "Une erreur interne est survenue."  
}); // Message pour l'utilisateur  
}
```

Principes appliqués :

- Messages d'erreur sans informations pour l'utilisateur côté client
- Logs détaillés en serveur pour le débogage

- Pas d'exposition de chemins, de versions, ou de détails techniques

Statut : Complété

Résultats et validation

6.1 Points obtenus

Catégorie	Tâches	Points
Obligatoires	8 tâches × 1 point	8 points
Optionnelles	5 tâches (1+1+1+2+2)	7 points
TOTAL		15 points

Détail des optionnelles :

1. HTTPS (1 point)
2. Politique de mot de passe fort (1 point)
3. Limitation des tentatives de login (1 point)
4. Verrouillage de compte (2 points)
5. Gestion des erreurs (2 points)

6.2 Failles de sécurité adressées (OWASP Top 10 2025)

Catégorie OWASP	Vulnérabilité	Solution implémentée
A01	Broken Access Control	JWT + rôles (admin/user)
A02	Cryptographic Failures	HTTPS, bcrypt avec salt/pepper
A03	Injection	Requêtes paramétrées (?)
A05	Access Control	Politique de mot de passe fort
A07	Identification & Authentication	JWT, rate limiting, verrouillage

Conclusion

Ce projet m'a permis de mettre en place une **sécurisation progressive** d'une application web, en couvrant les principes fondamentaux du **OWASP Top 10 2025**. J'ai également beaucoup appris sur la sécurité des applications web, un domaine qui m'intéresse beaucoup, car je compte m'orienter vers la **cybersécurité** plus tard. De plus, les tokens JWT m'ont été utiles dans le projet Passion-Lecture réalisé dans le cadre du module 295. Dans l'ensemble, ce fut un projet intéressant et utile pour la suite.

Date : 2026

Auteur : Thomas Reynaud

ETML - École Technique des Métiers de Lausanne